

Reviewer Pack — Geometric Arrangement Discrepancy Inverse Proportional to Di...

Entry 830213c96f89 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 17:17:07 UTC

Geometric Arrangement Discrepancy Inverse Proportional to Disjointness Communication Complexity

Entry ID: 830213c96f89 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 17:17:07 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For a random geometric arrangement of n pseudolines, the discrepancy of the communication matrix for the disjointness problem is $\Theta(1 / \log n)$, implying that the communication complexity is $\Omega(\log n)$.

Rationale (proposer's reasoning):

Geometric arrangements often exhibit structured discrepancy patterns, and their communication matrices may have inherent geometric constraints that influence the communication complexity of the disjointness problem.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9afb8bb85f75f731

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_pseudolines(n):
        pseudolines = []
        for i in range(n):
            x1, y1 = random.uniform(-1, 1), random.uniform(-1, 1)
            x2, y2 = random.uniform(-1, 1), random.uniform(-1, 1)
            if (x2 - x1) != 0:
                m = (y2 - y1) / (x2 - x1)
                c = y1 - m * x1
                pseudolines.append((m, c))
        return pseudolines

    def compute_discrepancy(pseudolines):
        n = len(pseudolines)
        discrepancy = 0
        for i in range(n):
            for j in range(i + 1, n):
                m1, c1 = pseudolines[i]
                m2, c2 = pseudolines[j]
                if m1 != m2:
                    x = (c2 - c1) / (m1 - m2)
                    y1 = m1 * x + c1
                    y2 = m2 * x + c2
                    if abs(y1 - y2) > 1e-6:
                        discrepancy += 1
        return discrepancy

    n = random.choice([5, 10, 15, 20, 30, 40])
    pseudolines = generate_pseudolines(n)
    discrepancy = compute_discrepancy(pseudolines)

    expected_discrepancy = 1 / math.log(n + 1)
    return {
        "metric_name": "discrepancy",
        "metric_value": discrepancy,
```

```

        "instances_tested": n * (n - 1) // 2,
        "conjecture_holds": abs(discrepancy - expected_discrepancy) < 0.1,
        "counterexample": "" if conjecture_holds else f"Discrepancy {discrepancy} does not match expected {expected_discrepancy}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) + [random.randint(100, 999) for _ in range(10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_discrepancy = sum(r["metric_value"] for r in results) / len(results)
    std_discrepancy = math.sqrt(sum((r["metric_value"] - mean_discrepancy) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_discrepancy} std={std_discrepancy} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no seeds tested")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df982df9.py", line 66, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df982df9.py", line 57, in run_trial
    "counterexample": "" if conjecture_holds else f"Discrepancy {discrepancy} does not match expected {expected_discrepancy}"
                        ~~~~~
NameError: name 'conjecture_holds' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'conjecture_holds', preventing result evaluation |
next: Fix the NameError in test code by properly initializing conjecture_holds variable

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109759
2	propose	ollama_remote	qwen3:8b	0	0	117772
3	preregistration	ollama_remote	qwen3:8b	0	0	26146
4	novelty	ollama_remote	qwen3:8b	0	0	22876
5	novelty	ollama_remote	qwen3:8b	0	0	14199
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15493
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9217
8	judge	ollama_remote	qwen3:8b	0	0	17526

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 332987 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in *research/audit/830213c96f89.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/830213c96f89.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/830213c96f89.tar.gz* (if generated)